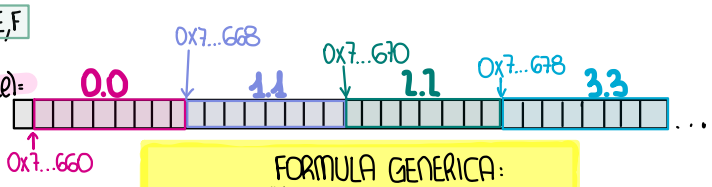
```
pcsdocker@THIS_IS_PCS_DOCKER:~/Data/ES_PCS_26/esercitazione_2/esercizio_1/build_es1$ ./esercizio1
VARIABILE X: 0
ARRAY DOUBLE:
indirizzo del componente 0 di ad:0x7ffc9ea58660
indirizzo del componente 1 di ad:0x7ffc9ea58668
indirizzo del componente 2 di ad:0x7ffc9ea58670
indirizzo del componente 3 di ad:0x7ffc9ea58678
ARRAY FLOAT:
indirizzo del componente 0 di af: 0x7ffc9ea58660
indirizzo del componente 1 di af: 0x7ffc9ea58668
indirizzo del componente 2 di af: 0x7ffc9ea58670
indirizzo del componente 3 di af: 0x7ffc9ea58678
indirizzo del componente 4 di af: 0x7ffc9ea58680
indirizzo del componente 5 di af: 0x7ffc9ea58688
indirizzo del componente 6 di af: 0x7ffc9ea58690
indirizzo del componente 7 di af: 0x7ffc9ea58698
ARRAY INT:
indirizzo del componente 0 di ai: 0x7ffc9ea58660
indirizzo del componente 1 di ai: 0x7ffc9ea58668
indirizzo del componente 2 di ai: 0x7ffc9ea58670
indirizzo variabile x: 0x7ffc9ea58630
indirizzo variabile y: 0x7ffc9ea5862c
pcsdocker@THIS_IS_PCS_DOCKER:~/Data/ES_PCS_26/esercitazione_2/esercizio_1/build_es1$
```

0x significa esadecimale → simboli 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

→ base

sizeof(double) = 8 byte



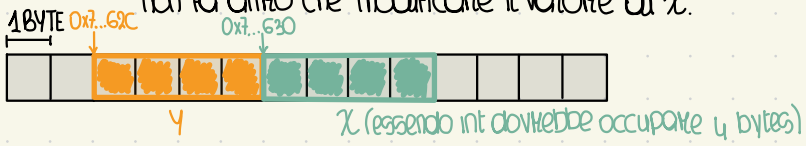
FORMULA GENERICA:
& array T[i] = base + i · sizeof[T]

→ base, sizeof(float)

→ base, size of(int)

PERCHÉ EGGE 0 INVECE CHE 1 ?

per gli indirizzi che sono usciti sappiamo che viene prima la y rispetto alla x.
la x si trova 4 bytes dopo la y. Noi però sappiamo che la y è un float e che sizeof(float) è esattamente 4.

0x7...630 −
0x7...62C =
0x0...004

=> Il codice in linea 12 sta dicendo:
vai all'indirizzo: (&y)[1] = base + 1 · sizeof(float) = 0x7...62C + 1 · 4 e metti il valore 0, ma quell'indirizzo di memoria è quello occupato dalla variabile x. Dunque il comando non fa altro che modificare il valore di x.